

MEMORIA ESCRITA DE MultiBioTraductor

(Documentación técnica)

1. Introducción

MultiBioTraductor es una aplicación web que permite traducir secuencias biológicas entre **ADN, ARN y proteína**, considerando diferentes estándares de código genético asociados a varios organismos modelo.

La herramienta está pensada para estudiantes y profesionales de biología, biotecnología y bioinformática que necesiten hacer traducciones rápidas y visualizar diferentes marcos de lectura (ORFs) sin recurrir a software pesado o scripts específicos.

2. Objetivos del proyecto

- Desarrollar una herramienta web accesible desde cualquier navegador moderno.
- Permitir la traducción:
 - ADN → ARN → Proteína
 - ARN → ADN → Proteína
 - Proteína → ARN → ADN (retrotraducción)
- Soportar:
 - Entrada de **una sola secuencia** por texto.
 - Entrada de **múltiples secuencias** mediante archivos **FASTA**.
- Ofrecer la descarga de resultados en **formato FASTA**, lista para integrarse con otros flujos de bioinformática.
- Incluir distintos estándares de codificación genética (“organismos”) seleccionables por el usuario.

3. Justificación

En el análisis de secuencias biológicas es habitual:

- Traducir genes a proteínas.
- Explorar ORFs alternativos (+1, +2, +3, -1, -2, -3).
- Procesar grandes lotes de secuencias en formato FASTA.

Existen herramientas en línea, pero muchas son pesadas, poco flexibles o dependen de servidores externos.

MultiBioTraductor propone:

- Una interfaz sencilla e intuitiva.
- Lógica de traducción ejecutada en el navegador (mayor privacidad de los datos).

- Manejo directo de FASTA y descarga de resultados.

4. Tecnologías utilizadas

- **Backend:**
 - Python 3
 - Flask (servido de plantillas y recursos estáticos)
- **Frontend:**
 - HTML5
 - CSS3 (archivos `base.css`, `traductor.css`, `unal.css`)
 - Bootstrap (diseño responsivo, modales, dropdowns)
 - JavaScript (lógica de negocio biológica y manejo de la interfaz):
`static/JS/script.js`.
- **Despliegue:**
 - `Dockerfile`
 - `docker-compose.yml` / `docker-compose-prod.yml`
 - `Procfile` (para PaaS tipo Heroku)
 - `requirements.txt` (gestión de dependencias).
- **Formato de datos:**
 - Entrada/salida en **FASTA**.

5. Estructura del proyecto

- `app.py`
Punto de entrada de la aplicación Flask y definición de rutas.
- `templates/`
 - `Base.html`: plantilla base (estructura HTML, inclusión de CSS/JS comunes).
 - `traductor.html`: vista principal de MultiBioTraductor.
 - (Opcionales) `Basecopy.html`, `traductor2.html`, etc.
- `static/`
 - `css/`: estilos generales y específicos del traductor.
 - `JS/script.js`: lógica de traducción, lectura de FASTA, generación de archivos y control de UI.
 - `bootstrap/`: recursos de Bootstrap (CSS y JS).
- Ficheros de configuración y despliegue:
`Dockerfile`, `docker-compose.yml`, `docker-compose-prod.yml`, `Procfile`, `requirements.txt`.

6. Arquitectura y diseño

6.1. Arquitectura general

La aplicación sigue un modelo **cliente-servidor ligero**:

- El **servidor Flask**:

- Renderiza las plantillas HTML.
- Sirve archivos estáticos (CSS, JS, Bootstrap).
- No realiza cálculos biológicos.
- El **cliente (navegador)**:
 - Ejecuta toda la lógica de traducción con JavaScript.
 - Procesa archivos FASTA.
 - Genera archivos de salida para descarga.

Esto reduce la carga del servidor y mantiene las secuencias siempre en el navegador del usuario.

6.2. Backend (Flask)

Ruta principal:

- **GET /** → renderiza `traductor.html`.
- **POST /** → actualmente solo imprime datos de prueba en consola (se deja preparado para futuras extensiones).

Ruta secundaria:

- **/salida** → plantilla `salida.html` (opcional, en esta versión no es esencial).

6.3. Frontend: plantillas

`traductor.html` (hereda de `Base.html`):

- Incluye:
 - Cabecera con el título **MultiBioTraductor** y un botón `?` que abre un **modal de ayuda**.
 - Sección “Secuencia a traducir”:
 - `textarea#secuencia`
 - `input#file` (solo `.fasta`), + etiqueta que muestra el nombre del archivo.
 - Sección “Opciones de traducción”:
 - Radios `tipoSecuencia`: ADN / ARN / Proteína.
 - `select#estandar`: lista de organismos.
 - `div#opcionesSalidas`: se rellena dinámicamente con checkboxes y botones según el tipo escogido.
 - Sección “Traducción”:
 - `div#salidaTraductor`: resultados de la secuencia escrita.
 - `div#salidaDocumento`: resultados de archivo FASTA.

Incluye al final el script principal:

```
<script src="{{ url_for('static', filename='JS/script.js') }}"></script>
```

7. Lógica de negocio (JavaScript)

Archivo: `static/JS/script.js`.

7.1. Datos globales

- `let datos = []`
Guarda las traducciones para poder generar posteriormente archivos FASTA.
Cada elemento:

`{ nombre: <string>, adn: <string>, arn: <string>, amino: <string> }`
- `const aminoacids_list`
Lista ordenada de aminoácidos (código una letra) y `*` como STOP.
- `let data = { 0: {...}, 1: {...}, ... 9: {...} }`
Diccionario de códigos genéticos:
 - `data[0]`: contiene todos los codones por aminoácido (ARN → proteína).
 - `data[1..9]`: codones preferentes para cada organismo (proteína → ARN).
- Diccionarios de conversión y complementarios:
 - `rna_to_dna, dna_to_rna`
 - `const complementario` (ADN)
 - complementarios ARN definidos en `complementarioARN`.

7.2. Flujo principal

7.2.1. `formulario()`

Responsabilidades:

1. Limpia la lista `datos`.
2. Lee la secuencia del `textarea`, la pasa a mayúsculas y elimina espacios/saltos de línea.
3. Valida:
 - Que se haya seleccionado un `tipoSecuencia`.
 - Que haya secuencia manual o archivo FASTA.
 - Que se haya elegido un `estandar`.
4. Si hay archivo:
 - Obtiene el `File` del input y llama a `loadFile(file)`.
5. Si no hay archivo:
 - Llama a `transformarsecuencia(secuencia, tipoSecuencia, estandar, "entrada")`.
6. Muestra el resultado devuelto en `#salidaTraductor`.
7. Habilita o deshabilita el botón **Descargar** según si hubo o no salida.

7.2.2. `loadFile(file)`

- Función asíncrona que:
 - Lee el archivo FASTA a texto.
 - Lo separa en líneas.

- Detecta encabezados > con regex.
- Reconstruye pares (`nombre`, `secuencia`).
- Para cada secuencia, llama a:
`transformarsecuencia(secuencia, tipoSecuencia, estandar, nombre)`
- Añade el HTML resultado a `#salidaDocumento`.

7.3. Transformación de secuencias

7.3.1. `transformarsecuencia(secuencia, tipoSecuencia, estandar, nombre)`

Función clave que:

1. Limpia la secuencia.
2. Según `tipoSecuencia`, realiza:
 - **AMINO:**
 - Valida que solo haya caracteres de aminoácidos y *.
 - `arn = aminoToRNA(secuencia, estandar)`
 - `dna = ARNtoADN(arn)`
 - Guarda en `datos`.
 - Crea un objeto `variables` con:
 - Nombre
 - Secuencia original
 - ADN forward, ADN complementario, ARN forward, ARN complementario (según checkboxes).
 - Devuelve `box(variables)`.
 - **ADN:**
 - Valida caracteres de ADN (incluyendo degenerados IUPAC).
 - `dna = secuencia`
 - `arn = ADNtoARN(dna)`
 - `aminoacido = ARNtoAMINO(arn, estandar)`
 - Guarda en `datos`.
 - Genera variables:
 - Complementario ADN
 - ARN forward y complementario
 - Proteínas ORF+1,+2,+3 y ORF-1,-2,-3 según opciones marcadas.
 - **ARN:**
 - Valida caracteres de ARN (incluyendo degenerados).
 - `arn = secuencia`
 - `adn = ARNtoADN(arn)`
 - `aminoacido = ARNtoAMINO(arn, estandar)`
 - Guarda en `datos`.
 - Genera variables:
 - ADN forward y complementario

- ARN complementario
- Proteínas ORF+1,+2,+3.

3. Habilita/deshabilita el botón **Descargar**.
4. Devuelve el HTML generado.

7.3.2. Funciones biológicas auxiliares

- `ARNtoAMINO(ARN, estandar=0, index=0)`
Traduce ARN a proteína:
 - Valida caracteres (AUGC).
 - Verifica que la longitud sea múltiplo de 3.
 - Recorre codones y los mapea a aminoácidos usando `data[estandar]` y `aminoacids_list`.
 - `index` es el desplazamiento inicial (marco de lectura).
- `ADNtoARN(ADN)` y `ARNtoADN(RNA)`
Reemplazan **T** ↔ **U** en la secuencia.
- `aminoToRNA(amino, estandar)`
Dada una proteína, genera ARN concatenando el codón asociado a cada aminoácido en `data[estandar]`.
- `complementarioADN(ADN)`
Devuelve la cadena complementaria usando el diccionario `complementario`.
- `complementarioARN(ARN)`
Genera el complementario en ARN (A↔U, C↔G).

7.4. Generación de archivos FASTA

- `generarDocumento(tipo, isReverse=false, marco=0, complementario=false)`
 - Recorre el arreglo global `datos`.
 - Según parámetros:
 - `tipo`: "adn", "arn", "amino".
 - `isReverse`: invierte o no.
 - `marco`: marco de lectura para proteína (0 = el que está almacenado, 1/2 para otros ORFs).
 - `complementario`: genera el complementario de ADN o ARN si aplica.

Compone un texto en formato:
>nombre

SECUENCIA

- Llama a `download("documento.fasta", texto)`.

- `download(filename, text)`
Crea un `<a>` temporal con `data:text/plain` y dispara la descarga.

8. Casos de uso

1. **Traducción ADN → proteína (ORFs forward y complementarios)**
 - Usuario pega secuencia de ADN.
 - Selecciona tipo **ADN** y organismo (por ejemplo, Homo sapiens).
 - Marca ORFs +1, +2, +3 y ORFs negativos si los necesita.
 - Obtiene ADN complementario, ARN y proteínas correspondientes.
2. **Retrotraducción proteína → ADN**
 - Usuario ingresa proteína en código una letra.
 - Selecciona tipo **Proteína** y el organismo deseado.
 - Obtiene ARN y ADN compatibles con ese código genético, junto con opciones de descarga FASTA.
3. **Traducción de lote FASTA**
 - Usuario carga un archivo `.fasta` con múltiples secuencias del mismo tipo.
 - Define opciones de salida.
 - Obtiene tarjetas con resultados individuales y un único archivo FASTA con las secuencias traducidas.

9. Pruebas realizadas

- **Pruebas de validación de entrada:**
 - Introducción de caracteres no válidos en ADN, ARN y proteína.
 - Verificación de los mensajes de alerta correspondientes.
- **Pruebas de traducción:**
 - Uso de codones conocidos (`AUG` → `M`, etc.).
 - Introducción de codones de parada y comprobación del símbolo `*`.
 - Secuencias de ARN de longitud no múltiplo de 3 para disparar el mensaje de error.
- **Pruebas de interfaz:**
 - Comportamiento del botón **Descargar** (activación tras traducción válida).
 - Carga de archivos FASTA con múltiples entradas.
 - Funcionamiento de los checkboxes de salida y el recálculo de resultados.

10. Conclusiones

- MultiBioTraductor cumple el objetivo de ofrecer una herramienta web ligera para la traducción de secuencias entre ADN, ARN y proteína.
- La lógica en el cliente mejora la privacidad de las secuencias y reduce carga al servidor.
- El soporte de FASTA y ORFs múltiples facilita tareas didácticas y de análisis preliminar en bioinformática.
- Como trabajo futuro se plantea:
 - Añadir más códigos genéticos (ej. mitocondrial).
 - Permitir la definición de estándares personalizados.

- Incorporar una API o backend para guardar historiales y realizar análisis más complejos.

11. Licencia y autores

Este proyecto se distribuye bajo la **Licencia MIT** (permisiva). Puedes usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del software, siempre que mantengas el aviso de copyright y esta nota de licencia en las copias o partes sustanciales del software. No ofrece garantías.

- **Autora principal:** *Hinara Pastora Sánchez Mata*. Información de contacto: 3044822338, hisanchezm@unal.edu.co
- **Crédito institucional:** “Desarrollado por la línea de interacciones moleculares — **Biología Funcional**”.